

Spring Boot — Lecture 1

Introduction to Spring Framework

1. About This Series

- Beginner-friendly — every topic from scratch to advanced, with depth.
- Not just Spring Boot syntax — covers Spring Core, Spring MVC internals too.
- Practical industry knowledge + interview preparation throughout.
- Notes + practice questions shared after every lecture on Git channel.

Why can't you just learn Spring Boot alone? Because Spring Boot sits on top of Spring Core, Spring MVC, Spring Data etc. Without knowing the internals you can't debug, scale, or crack interviews properly.

2. Client-Server Architecture

When you visit amazon.in, your browser sends a request to Amazon's server (a computer hosting their application). The server processes it and sends back a response which the browser displays. This is the **Client-Server Model**.

Entity	Role	Examples
Client	Sends HTTP request	Browser, Mobile App, React Frontend, Postman, Another Microservice
Server	Receives, processes, responds	Spring Boot App running on a machine / cloud

A server can also act as a client — e.g., Order Service calling Payment Service in a microservices setup.

3. HTTP — HyperText Transfer Protocol

Protocol = a rulebook / standard. HTTP defines how client and server communicate — the format of requests, responses, and how data travels over the internet.

HTTP vs HTTPS

Protocol	Encryption	Port
HTTP	None — plain text (can be intercepted)	80
HTTPS	Encrypted end-to-end (S = Secured)	443

HTTP Methods

Method	Purpose	Body?
GET	Retrieve / fetch data	No
POST	Create / store new data	Yes
PUT	Update entire resource	Yes
PATCH	Update specific field	Yes
DELETE	Remove a resource	No

4. HTTP Request Structure (4 Parts)

Part	Description	Example
------	-------------	---------

Method	Type of operation	GET / POST / PUT / DELETE
URL / Path	Host + endpoint	www.coderarmyarmy.in/courses
Headers	Key-value metadata (optional)	Accept: application/json Content-Type: application/json
Body	Data payload (POST/PUT only)	{ "email": "a@b.com", "password": "xyz" }

GET example (no body):

```
GET www.coderarmyarmy.in/courses Accept: application/json
```

POST example (login — has body):

```
POST www.coderarmyarmy.in/login Content-Type: application/json { "email": "user@gmail.com", "password": "secret123" }
```

5. HTTP Response Structure (3 Parts)

Part	Description	Example
Status Code	Indicates success/failure	200, 201, 404, 500, 503
Headers	Metadata about response	Content-Type: application/json
Body	Actual returned data	{ "message": "Login successful" }

Common Status Codes

Code	Meaning
200	OK — success
201	Created — new resource created
404	Not Found — endpoint/resource doesn't exist
500	Internal Server Error — server crashed
503	Service Unavailable

6. How Java Handles Web Requests — Evolution

6.1 Problem with raw Java (java.net package)

Java has had networking support since v1 via the **java.net** package (ServerSocket, Socket classes). You could listen on a port — but everything had to be manual:

- Read raw input stream (stream of bytes — JVM has no idea it's HTTP)
- Manually parse the HTTP request (extract method, URL, headers, body)
- Manually map URL endpoint to the right Java method (if/else chains)
- Manually build the HTTP response
- Manually implement multi-threading to handle concurrent users

This is huge boilerplate — no developer wants to write it every time.

6.2 Servlet + Servlet Container (1997)

Introduced as part of Java EE. A **Servlet** is a Java class that runs inside a **Servlet Container** (also called a Web Server).

Famous Servlet Containers	Role
Tomcat (most popular)	Manages port, reads bytes, converts to HTTP request, calls the right Servlet, builds HTTP response, handles threads
Jetty	Lightweight alternative
Undertow	Used by WildFly / JBoss

Tomcat does all the boilerplate automatically. Your job: just write business logic inside a Servlet. Tomcat stays alive 24/7, listens for requests, and calls the correct Servlet based on the URL.

Normal Java program vs Web Server flow:

Normal Java App	Web Server (Tomcat)
Start → Execute instructions → Exit	Start → Wait for request → Give response → Keep running (repeat forever)

A website is not a program that runs once. It stays up 24/7 listening for requests.

7. Why Spring Framework?

Servlet-based code worked, but at enterprise scale it became **tightly coupled** — many objects interconnected, making the app hard to scale and test. Spring introduced concepts that made apps **loosely coupled**:

- Dependency Injection (DI)
- Inversion of Control (IoC)
- Beans — managed objects

Spring is an Ecosystem — not just one framework:

Layer	Module	Purpose
Base (Core)	Spring Core	DI, IoC, Beans — foundation of everything
Web	Spring MVC	Build REST APIs / web apps (uses Servlets internally)
Data	Spring Data / JPA	Connect to databases (uses Hibernate → JDBC internally)
Security	Spring Security	Authentication, Authorization, Login
Cross-cutting	Spring AOP	Aspect Oriented Programming (logging, transactions)
Top layer	Spring Boot	Automation layer — auto-configures everything above

Spring Boot ≠ replacement for Spring Framework

*Spring Boot is an **opinionated automation layer** built on top of all the modules above. It pre-configures everything so you can start writing business logic immediately. But to customize or debug it, you **MUST** understand the internals (MVC, Security, Data, AOP). Spring Boot just speeds up bootstrapping — the core skills remain Spring MVC, Spring Data etc.*

8. Database Layer Evolution

Technology	What you write	Notes
------------	----------------	-------

JDBC	Raw SQL queries in Java code	Manual, verbose, but direct
JPA (spec)	Method calls, no SQL	Just a specification/rulebook, not an implementation
Hibernate	Implements JPA, uses JDBC internally	Most popular JPA implementation
Spring Data	Repository interfaces, zero boilerplate	Built on Hibernate → JPA → JDBC

Chain: Spring Data → Hibernate → JPA → JDBC → Database. You need to understand the full chain to debug issues properly.

9. Complete Spring Boot Architecture

Layer	Technology	Job
Client	Browser / App / Postman	Send HTTP request
Web Layer	Spring MVC (Tomcat inside)	Route request to correct controller method
Business Layer	Your Java code / Services	Business logic
Security	Spring Security	Authenticate / Authorize
Data Layer	Spring Data → Hibernate → JDBC	Talk to database
Database	MySQL / Oracle / PostgreSQL	Store / retrieve data

Monolithic vs Microservices:

Architecture	Description
Monolithic	One single Spring Boot app handles all endpoints
Microservices	Multiple small Spring Boot apps (Order Service, Payment Service, User Service) that call each other via HTTP

Next Lecture: Building First Spring Boot Hello World App + Spring Initializr